

SMALL PORTFOLIO PROJECT 01

Virtual Signal Conditioner & Log Player

Hardware-free, real-time IMU signal conditioning: stream a public dataset at sensor speed, profile its spectrum, and clean it with a hand-derived digital filter.

Difficulty: Beginner+ (weekend scale) **Est. Effort:** 3–4 focused days **Stack:** C++17, CMake, Linux

1 Pre-Beginning Research Readings

Before writing a single line of code, build fluency in three domains that intersect in this project: the **mathematics of sampling and filtering**, the **systems programming of deterministic timing loops**, and the **structure of real IMU datasets**. Treat the list below as a guided curriculum, not a bibliography — read with a pen and reproduce the key derivations yourself.

1.1 Mathematics & Signal Theory

- **Oppenheim & Schaffer, *Discrete-Time Signal Processing* (3rd ed.).** The canonical reference.
 - *Ch. 4 — Sampling of Continuous-Time Signals:* Nyquist–Shannon theorem, aliasing, and reconstruction. You must be able to explain why a 200 Hz stream cannot faithfully represent content above 100 Hz.
 - *Ch. 7 — Filter Design Techniques:* concentrate on the **bilinear transform** (§7.2) and the analog-to-digital mapping of Butterworth prototypes into IIR difference equations.
- **Lyons, *Understanding Digital Signal Processing* (3rd ed.).** A gentler, intuition-first companion. Read the chapters on the DFT, spectral leakage, and windowing (Hann vs. rectangular) — you will need windowing to get a clean spectrum.
- **Julius O. Smith III, *Introduction to Digital Filters* (CCRMA, Stanford — free online).** Read [the sections on the Bilinear Transform and Direct-Form I / II topologies](#). The companion *Mathematics of the DFT* is the clearest free derivation of the transform you will implement.
- **Press et al., *Numerical Recipes* (3rd ed.), Ch. 12–13.** The FFT (Cooley–Tukey radix-2), the structure of a real-input transform, and power-spectrum estimation. Read this before writing your own FFT.

1.2 Systems Programming & Deterministic Timing

- **Linux man pages:** `clock_gettime(2)` and `nanosleep(2)`. Understand the difference between `CLOCK_REALTIME` (wall clock, can jump) and `CLOCK_MONOTONIC` (never steps backward) — and why monotonic time is mandatory for a fixed-rate loop.
- **cppreference:** `std::chrono` (`clocks`, `durations`, `time_points`) and `std::this_thread::sleep_until`. Learn why `sleep_until(next_deadline)` accumulates far less drift than `sleep_for(period)`.
- **Williams, *C++ Concurrency in Action* (2nd ed.), Ch. 1–2.** Not strictly required here but the “time facilities” discussion underpins everything you will do in the Large projects.

1.3 Datasets & Domain Specifications

- **Burri et al., “The EuRoC micro aerial vehicle datasets” (IJRR, 2016).** Read the sensor description: the ADIS16448 IMU, its 200 Hz rate, and the structural vibration noise injected by the multirotor. Inspect the raw `imu0/data.csv` layout — timestamp (ns), three angular velocities (rad/s), three linear accelerations (m/s²).
- **Geiger et al., “Vision meets Robotics: The KITTI Dataset” (IJRR, 2013).** An alternative ground-vehicle source; skim the OXTS IMU/GPS format so your parser can target either dataset.
- **Allan variance primer (IEEE-STD-952 background, or any “IMU noise model” tutorial).** Optional but valuable: it explains *why* the noise you are filtering looks the way it does (white noise + bias instability).

2 Detailed Project Description

Project Goal

Construct a hardware-free, command-line signal-conditioning pipeline that **replays a public IMU dataset at true sensor rate**, identifies its noise spectrum with a self-written FFT, and attenuates structural vibration with a **hand-derived Butterworth IIR filter** — with no external DSP libraries doing the mathematics for you.

2.1 Theory You Will Implement

The continuous Butterworth low-pass prototype of order N has the magnitude response

$$|H(j\Omega)|^2 = \frac{1}{1 + \left(\frac{\Omega}{\Omega_c}\right)^{2N}}, \quad (\text{2nd order}) \quad H(s) = \frac{\Omega_c^2}{s^2 + \sqrt{2}\Omega_c s + \Omega_c^2}.$$

You map it to the digital domain with the **bilinear transform** (with frequency pre-warping so the cutoff lands where you intend):

$$s = \frac{2}{T} \frac{1 - z^{-1}}{1 + z^{-1}}, \quad \Omega_c = \frac{2}{T} \tan\left(\frac{\omega_c T}{2}\right).$$

The result is a Direct-Form I difference equation, which is the only thing your real-time loop actually executes per sample:

$$y[n] = \frac{1}{a_0} \left(b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-2] \right).$$

A 2nd-order filter rolls off at $6N = 12$ dB/octave above ω_c — which is exactly the attenuation your acceptance test will measure.

2.2 Core Deliverables

1. **Log Ingestion Module.** A streaming C++ parser that reads EuRoC/KITTI CSV (or binary) logs *line-by-line*, never loading the whole file into the heap. Exposes one sample at a time via an iterator-like interface.
2. **Deterministic Real-Time Player.** A steady loop built on `std::chrono::steady_clock + sleep_until` that emits samples at exactly 200 Hz ($\Delta t = 5$ ms), honouring the dataset's own timestamps where present.
3. **Analytical FFT Profiler.** A windowed (Hann) radix-2 FFT over a sliding buffer that prints the dominant noise frequencies, so you can *justify* your chosen cutoff instead of guessing it.
4. **Custom IIR Butterworth Filter.** A pure-math 2nd-order low-pass implemented from the coefficients you derived above — no SciPy, no `<dsp>` library. Cutoff and order configurable at runtime.
5. **Validation Harness.** Side-by-side raw / filtered CSV output plus a short plotting script (Gnuplot or matplotlib) producing a time-series and a spectrum-before/after figure.

2.3 Suggested Timeline

Day	Theme	Outcome
Day 1	Ingestion & timing	Parser streams a dataset; 200 Hz loop prints clean samples with measured jitter.
Day 2	Spectral analysis	FFT profiler maps the noise spikes; you pick and justify a cutoff.
Day 3	Filter math	Bilinear coefficients derived by hand; Direct-Form I loop runs in real time.
Day 4	Validation & write-up	Raw vs. filtered CSV + plots; short README explaining the result.

2.4 Acceptance Criteria

Acceptance Criteria — Definition of Done

- **Timing precision.** Over 10,000 consecutive iterations the 200 Hz loop holds a per-frame error within ± 0.5 ms; report the measured mean and standard deviation of the interval.
- **Filter correctness.** High-frequency content above the chosen cutoff is attenuated by at least the theoretical 12 dB/octave, verified against a reference response (a 10-line SciPy/MATLAB script is allowed *for checking only*).
- **Memory discipline.** The steady-state loop performs *zero* dynamic allocations; the program passes Valgrind/ASan with no leaks or invalid accesses.
- **Reproducibility.** A fresh clone builds with one `cmake` invocation and runs against a documented dataset path without code edits.

Common Pitfalls

- Using `sleep_for(period)` instead of `sleep_until(deadline)` — it lets scheduling latency accumulate into seconds of drift.
- Forgetting to **pre-warp** the cutoff before the bilinear transform, so the digital cutoff lands at the wrong frequency.

- Running the FFT on a rectangular window and mistaking spectral leakage for real noise peaks.

3 Grading System

Criterion	Pts	Evaluated by
Loop determinism & timing accuracy	25	Std. dev. of measured loop intervals.
Mathematical accuracy of the IIR filter	35	Output vs. reference response; correct coefficients.
Memory & systems discipline	20	No heap allocs in loop; clean Valgrind/ASan.
Analytical verification & logging	20	Plots/CSV that clearly demonstrate attenuation.
Total	100	

Stretch goals (bonus, portfolio polish). Add a real-time spectrogram view; support multi-channel filtering across all six IMU axes; benchmark Direct-Form I vs. II for numerical stability in fixed precision; expose the cutoff over a tiny CLI so a grader can sweep it live.

4 Goal, Learning Outcomes & Conquered Techniques

4.1 Overall Goal

Bridge raw data parsing with real-time systems programming, proving you can implement low-level mathematical processing pipelines that filter real-world physical data *deterministically* and *without microcontrollers*.

4.2 Key Learning Outcomes

You will understand how physical vibration corrupts robotic state estimates, how the bilinear transform turns a continuous filter into executable code, and how deterministic timing constructs let ordinary Linux processes mimic hardware sample rates.

4.3 Techniques Conquered

- High-precision loop rate-limiting with `std::chrono + sleep_until`.
- Streaming parsers for large, high-frequency log payloads.
- Manual IIR filter design via the pre-warped bilinear transform (Direct-Form I).
- Windowed FFT spectral analysis and cutoff justification.
- Memory and performance profiling with Valgrind / AddressSanitizer.

4.4 How to Frame This on Your Resume / GitHub

“Built a hardware-free C++17 real-time IMU conditioning pipeline that replays public datasets (EuRoC/KITTI) at 200 Hz with sub-millisecond loop jitter, performs windowed-FFT spectral analysis, and applies a hand-derived 2nd-order Butterworth IIR filter — zero

steady-state heap allocation, verified under Valgrind.” Pin the repo with the before/after spectrum figure as the README hero image.